

Ejercicio 3: Reparto de Trabajos en Dos Máquinas (Makespan Mínimo)

Este problema es un clásico de programación dinámica. El enunciado intenta distraerte con asimetrías de tiempos de procesamiento, pero la idea real es: **cada trabajo va COMPLETO a la máquina A o COMPLETO a la máquina B** y queremos minimizar el tiempo total de finalización (*makespan*).

1. Qué significa "tiempo mínimo"

Si definimos:

- T_A : Tiempo acumulado de procesamiento en la máquina A .
- T_B : Tiempo acumulado de procesamiento en la máquina B .

Como las máquinas trabajan en paralelo, el tiempo total del sistema es:

$$\max(T_A, T_B)$$

El proceso completo termina cuando acaba la máquina más lenta.

2. Qué estamos decidiendo

Para cada trabajo $i \in \{1, \dots, N\}$ debemos tomar una decisión binaria:

- O lo mandamos a la máquina A (añadiendo a_i a su carga).
- O lo mandamos a la máquina B (añadiendo b_i a su carga).

Esto es realmente un **problema de partición** donde queremos repartir los trabajos de forma que equilibremos las cargas de ambas máquinas.

3. Estado de la Programación Dinámica

Definimos la función $f(i, x)$ como:

$$f(i, x) = \begin{array}{l} \text{El mínimo tiempo requerido en la máquina } B \text{ para procesar una} \\ \text{selección de los primeros } i \text{ trabajos, dado que la máquina } A \\ \text{ha acumulado exactamente } x \text{ unidades de tiempo en total.} \end{array}$$

4. Inicialización

Al no haber considerado ningún trabajo ($i = 0$):

- Si la máquina A lleva un tiempo de 0, la máquina B también requiere un tiempo de 0:

$$f(0, 0) = 0$$

- Cualquier otra carga $x > 0$ en la máquina A es imposible de obtener sin trabajos:

$$f(0, x) = \infty \quad \forall x \neq 0$$

5. Recurrencia

Para calcular el estado del trabajo i con una carga x en la máquina A , evaluamos las dos decisiones de transición posibles:

1. **Mandarlo a la máquina A :** La máquina A acumula x tras sumarle a_i . Por tanto, el estado previo en la máquina A debía ser $x - a_i$. La máquina B no añade tiempo nuevo para este trabajo:

$$f(i, x) = f(i - 1, x - a_i) \quad (\text{sólo si } x \geq a_i)$$

2. **Mandarlo a la máquina B :** La máquina A mantiene su carga x sin cambios. El tiempo acumulado en la máquina B aumenta en b_i unidades de tiempo:

$$f(i, x) = f(i - 1, x) + b_i$$

Uniendo ambas opciones, la relación de recurrencia que minimiza el tiempo en B es:

$$f(i, x) = \min \left(f(i - 1, x - a_i), f(i - 1, x) + b_i \right)$$

6. Pseudocódigo Modo Examen

A continuación se presenta el algoritmo simplificado para rellenar la tabla de estados y calcular el coste óptimo:

```

Funcion CalcularAsignacionTrabajos(N, a, b)
Inicio
    // Calcular el límite máximo de tiempo para la máquina A
    SUMA_A <- 0
    Para i <- 1 Hasta N Con Paso 1 Hacer
        SUMA_A <- SUMA_A + a[i]
    FinPara

    // Crear la tabla DP para albergar la función f(i, x)
    Crear matriz f de dimensiones (N + 1) x (SUMA_A + 1)

    // Inicialización de casos base
    f[0, 0] <- 0
    Para x <- 1 Hasta SUMA_A Con Paso 1 Hacer
        f[0, x] <- Infinito
    FinPara

    // Llenar la tabla iterativamente
    Para i <- 1 Hasta N Con Paso 1 Hacer
        Para x <- 0 Hasta SUMA_A Con Paso 1 Hacer
            f[i, x] <- Infinito

```

```

// Opción 1: Mandar el trabajo i a la máquina B
f[i, x] <- Minimo(f[i, x], f[i - 1, x] + b[i])

// Opción 2: Mandar el trabajo i a la máquina A
Si x >= a[i] Entonces
    f[i, x] <- Minimo(f[i, x], f[i - 1, x - a[i]])
FinSi
FinPara

// Búsqueda del Óptimo Global
mejor_makespan <- Infinito
Para x <- 0 Hasta SUMA_A Con Paso 1 Hacer
    mejor_makespan <- Minimo(mejor_makespan, Maximo(x, f[N, x]))
FinPara

Retornar mejor_makespan
FinFuncion

```

7. Análisis de Complejidad

Sea $M = \sum_{i=1}^N a_i$ la suma total de los tiempos en la máquina A (cota superior de carga):

- **Complejidad Temporal:** $\Theta(N \cdot M)$. La tabla contiene exactamente $N \cdot M$ estados individuales. Para calcular cada estado, realizamos operaciones de asignación y cálculo del mínimo en tiempo constante $O(1)$. El coste de búsqueda del mínimo óptimo final es de $O(M)$ y la reconstrucción hacia atrás es de $O(N)$. Por tanto, el coste total en tiempo es pseudopolinómico.
- **Complejidad Espacial:** $\Theta(N \cdot M)$ para almacenar la matriz bidimensional f necesaria para poder recuperar el reparto de trabajos óptimo en la fase de reconstrucción.